Data Structures and Algorithms - 3 (25CS2103E)

# Full-Text Search Engine on a Suffix-Array Index


**Team Members**

Name : Sai Saketh Roll No: 2520090124

Name : Sonu Paswan Roll No: 2520090204

**Under the Esteemed Guidance of**

Mrs. CH Anitha

DSA-3 Faculty | Department of CSIT

# Contents

1. Abstract
2. Introduction
3. Literature Survey
4. Problem Statement and Objectives
5. Flow Chart
6. Results
7. Hardware and Software Requirements
8. Conclusion
9. Future Scope

# Abstract

The Full-Text Search Engine on a Suffix-Array Index is designed to provide fast and efficient searching across a collection of documents. The system builds a searchable index using a Suffix Array, which stores the starting positions of all suffixes of a text in lexicographically sorted order. This enables substring, prefix, and phrase queries to be processed using binary search instead of scanning every document sequentially.

The project implements an $O(n \log^2 n)$ suffix-array construction algorithm and uses Kasai's algorithm to construct the Longest Common Prefix (LCP) array. LCP information is used for LCP-accelerated binary search, reducing unnecessary character comparisons. The system also supports ranked search results. Performance is evaluated using query latency, index size, construction time, and memory usage, with a comparison against a traditional inverted index. The project demonstrates the practical use of data structures and string-processing algorithms for efficient information retrieval.

# Introduction

Searching large document collections efficiently is an important information-retrieval problem.

Sequential scanning checks documents one by one and can become slow as the collection grows.

A Suffix Array provides a compact index for locating patterns in text.

Suffixes are stored in lexicographical order, allowing binary-search-based pattern matching.

Kasai’s algorithm is used to construct the Longest Common Prefix (LCP) array.

LCP-accelerated binary search reduces repeated character comparisons during searching.

The system supports substring, prefix, and phrase queries and produces ranked results.

Performance is compared with a traditional inverted-index approach.

# Literature Survey

| Technique | Main Idea | Limitation / Observation |
|---|---|---|
| Sequential Search | Scans documents or text positions one by one. | Can be slow for large collections. |
| Inverted Index | Maps terms to documents or positions containing them. | Very effective for term-based search; substring handling is less direct. |
| Suffix Tree | Represents suffixes in a tree structure for pattern matching. | Can require high memory. |
| Suffix Array | Stores sorted starting positions of all suffixes. | Needs additional structures/logic for efficient query processing. |
| LCP Array | Stores common-prefix lengths between adjacent suffixes. | Works with the suffix array to reduce repeated comparisons. |

**Key observation: Suffix arrays offer a compact indexing approach, while LCP information can improve search efficiency.**

# Problem Statement and Objectives

Problem Statement:
Develop an efficient full-text search system for a document collection that supports substring, prefix, and phrase queries while reducing query-processing time through suffix-array indexing and LCP-accelerated binary search.

## Objectives

Build a searchable index using a Suffix Array.

Implement $O(n \log^2 n)$ suffix-array construction.

Construct the LCP array using Kasai's algorithm.

Implement LCP-accelerated binary search.

Support substring, prefix, and phrase queries.

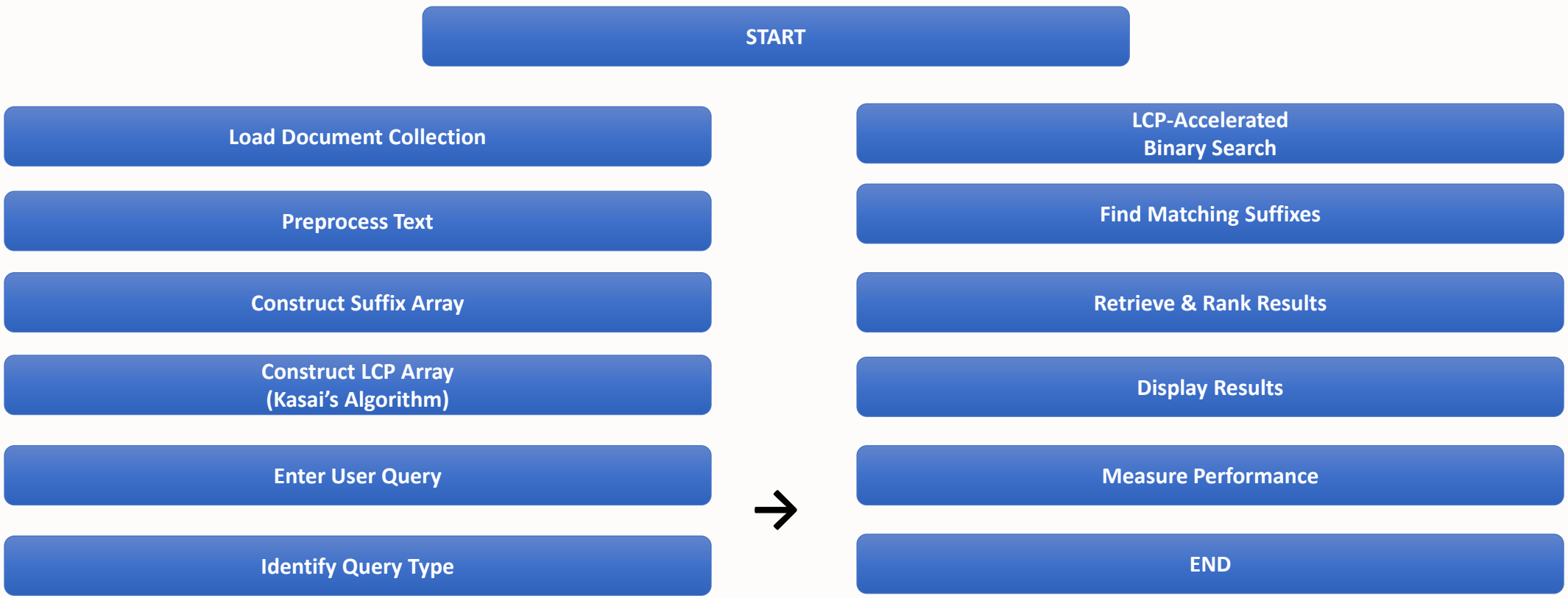
Provide ranked search results.

Measure query latency, construction time, memory usage, and index size.

Compare the suffix-array approach with an inverted index.

# Flow Chart

## Process Flow



# Results

## Performance parameters to be recorded during implementation

| | |
|---|---|
| **Query Latency** | Time taken to process substring, prefix, and phrase queries. |
| **Index Construction Time** | Time required to build the suffix array and LCP array. |
| **Index Size** | Storage/memory required by the searchable index. |
| **Search Correctness** | Whether all expected matching documents/positions are retrieved. |
| **Ranking Quality** | Whether relevant documents appear before less relevant results. |

**Important: enter the actual measured values from the implemented program; no experimental numbers are assumed here.**

# Hardware and Software requirements

## Hardware Requirements

Processor: Intel Core i3 or higher

RAM: 4 GB or higher

Storage: At least 500 MB free space

Keyboard and Mouse

Monitor / Display

## Software Requirements

Operating System: Windows / Linux / Ubuntu

Programming Language: Java

IDE: IntelliJ IDEA / Eclipse / VS Code

Java Development Kit (JDK)

Input: Text / document collection

Output: Ranked search results

**Core concepts: Suffix Array • LCP Array • Kasai's Algorithm • Binary Search • String Processing • DSA**

# Conclusion

The project demonstrates how a Suffix Array can be used to build an efficient full-text search index.

Lexicographically sorted suffix positions enable binary-search-based pattern matching.

Kasai’s algorithm provides the LCP array for neighboring suffixes.

LCP-accelerated binary search reduces repeated character comparisons.

The system is designed to support substring, prefix, and phrase queries.

Ranked results help present relevant documents in a useful order.

Evaluation focuses on query latency, index size, construction time, and memory usage.

Comparison with an inverted index highlights the strengths and trade-offs of both approaches.

# Future Scope

Support very large document collections with improved indexing techniques.

Optimize suffix-array construction for larger datasets.

Improve result ranking using more advanced relevance measures.

Add case-insensitive, fuzzy, and typo-tolerant searching.

Support multiple languages and different document formats.

Develop a graphical or web-based search interface.

Explore parallel or distributed index construction.

Add efficient document insertion and deletion.

Use caching to improve repeated-query performance.

Compare with additional modern information-retrieval and indexing techniques.

(DEEMED TO BE UNIVERSITY)

# THANK YOU